

FitTrack AI

Personalized Fitness Recommendations Powered by AI

An AI-powered fitness tracking and recommendation REST API / backend project

Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt.js · Google Gemini AI

Table of Contents

2. Abstract

Many people record workouts by hand or not at all, and they rarely receive guidance that fits their own goals and experience. FitTrack AI is a backend REST API that addresses this problem. It is built with Node.js, Express.js and MongoDB Atlas (through Mongoose). Users register and log in, and the API protects private routes with JSON Web Tokens (JWT), while passwords are stored only as bcrypt hashes. Authenticated users can add, view, update, delete and search their own workouts. Two AI endpoints use Google Gemini: one generates a workout recommendation from a user's age, fitness goal and experience, and the other generates fitness insights from summary figures (total workouts, average duration and total calories burned). The project uses a layered architecture (routes, middleware, controllers, services and models), input validation, centralized error handling and a standard JSON response format. It is a backend/API project and was tested with Thunder Client / Postman.

3. Introduction

Regular exercise is easier to sustain when progress is recorded and when the plan suits the person following it. Tracking usually needs a dependable place to store workout data, and guidance needs some understanding of a person's goals and experience. FitTrack AI combines the two in a REST API: a secure workout tracker and a Gemini-powered assistant that produces general recommendations and insights.

The project follows the standard backend approach: JSON over HTTP, token-based authentication, a cloud database and a modular code structure. Because it is an API, any client that can send HTTP requests can use it. During development and testing, Thunder Client / Postman acts as that client.

4. Problem Statement

- Manual tracking in notebooks or loose notes is easy to forget, hard to search and easy to lose.
- Without a structured record (workout name, category, duration, calories, date), it is difficult to review past training.
- General fitness advice often ignores the individual's age, goal and experience level.
- Personal fitness data needs to be stored securely and shown only to its owner.
- Interpreting totals such as workout count, duration and calories takes effort without a summary.

5. Objectives

1. Provide secure user authentication (registration, login and JWT-protected routes).
2. Allow users to add, view, update and delete their workouts.
3. Allow users to search their workouts.
4. Generate AI-based workout recommendations with Google Gemini.
5. Generate AI-based fitness insights from workout summary data.
6. Expose all functions as structured REST APIs with consistent JSON responses.
7. Keep each user's workout data private and protect sensitive configuration such as passwords and API keys.

6. Existing System

Basic or manual workout tracking, such as paper logs, notes or simple spreadsheets, generally has these limitations. This section makes no claims about any particular commercial application.

- Data entry is unstructured and inconsistent.
- Searching or reviewing earlier workouts is slow.
- Data is not protected by user accounts, or is available only on one device.
- There is no personalized recommendation based on a person's goal and experience.
- Any analysis of progress has to be done by the user.

7. Proposed System

FitTrack AI provides a secure, API-based system. Users create an account and log in to receive a JWT. With that token they manage workouts stored in MongoDB Atlas, each linked to its owner. They can search their workouts and can request AI output: a recommendation from age, fitness goal and experience, or insights from total

workouts, average duration and total calories burned. Requests are validated, all errors go through a centralized handler, and every response follows the same JSON format. Secrets (MongoDB connection string, JWT secret and Gemini API key) are held in environment variables.

8. Requirements

8.1 Functional Requirements

- Register a new user account and log in with email and password.
- Issue a JWT on login and verify it on protected routes.
- Return the authenticated user's profile.
- Create, list, view by ID, update and delete workouts belonging to the authenticated user.
- Search workouts with the `q` query parameter.
- Return an AI workout recommendation and AI fitness insights.
- Validate input and return clear JSON errors.

8.2 Non-Functional Requirements

- **Security:** passwords stored as bcrypt hashes, JWT-protected routes, secrets kept in environment variables.
- **Maintainability:** modular structure with separate routes, middleware, controllers, services and models.
- **Reliability:** centralized error handling gives consistent handling of failures.
- **Consistency:** standard JSON API responses.
- **Availability of cloud services:** depends on MongoDB Atlas and the Gemini API being reachable.

8.3 Hardware Requirements

	Minimum	Recommended
Computer / laptop	Dual-core 64-bit processor	Quad-core processor
RAM	4 GB	8 GB or more
Storage	10 GB free	256 GB SSD, 20 GB or more free
Internet	Stable connection (needed for MongoDB Atlas, the Gemini API, npm and GitHub)	

8.4 Software Requirements

Windows / Linux / macOS	Operating system
Node.js (LTS) and npm	Runtime and package manager
Visual Studio Code (or equivalent)	Code editor
MongoDB Atlas	Cloud database
Thunder Client / Postman	API testing
Git / GitHub	Version control

Development technologies: JavaScript, Node.js, Express.js, Mongoose, JWT, bcrypt.js and the Google Gemini API.

9. System Design

9.1 Technical Architecture

FitTrack AI uses a layered architecture. A request from the client passes through the Express server, the API routes and the authentication middleware to a controller. The controller calls a service, which uses the Mongoose models to reach MongoDB Atlas. For AI requests, the controller calls the Gemini AI service, which calls the Google Gemini API. Figure 1 shows the flow.

FitTrack AI – Technical Architecture Diagram Client / Thunder Client or PostmanHTTP requests + JWT, JSON responses Express.js ServerJSON parser · CORS · Morgan · config API Routes/api/auth · /api/workouts · /api/ai Authentication MiddlewareVerify JWT · attach user to request ControllersValidate input · call services · respond ServicesBusiness logic Mongoose ModelsUser · Workout MongoDB AtlasCloud database AI LAYER Gemini AI ServicePrompt building · API call Google Gemini APIExternal AI model AI Recommendation / InsightReturned to the controller Client API / server Authentication Business logic Database AI Figure 1: FitTrack AI – Technical Architecture Diagram

9.2 Database / ER Design

The database has two collections, User and Workout. One User has many Workouts; the `user` field of a Workout stores the `_id` of its owner (a Mongoose reference to User). No other entities exist.

FitTrack AI – Entity Relationship Diagram
USER `_id` : ObjectIdPK name : Stringemail : String (unique)password : String (hashed)createdAt : DateupdatedAt : Date
WORKOUT `_id` : ObjectIdPK workoutName : Stringcategory : Stringduration : NumbercaloriesBurned : NumberworkoutDate : Date user : ObjectIdFK → USER.`_id` createdAt : DateupdatedAt : Date has many 1N PK = Primary Key FK = Foreign Key / reference
USER 1 — N WORKOUT
Figure 2: FitTrack AI – Entity Relationship Diagram

9.3 User Flow

A new user registers, then logs in to obtain a JWT. The token is sent with each protected request. If it is valid, the user can view the profile, manage and search workouts, or request AI output. Errors at any stage return a JSON error response.

FitTrack AI – User Flow Diagram
Start Has account? No Registervalidate · hash password · save user Yes Login (email + password) Validcredentials? No 401 error JSON response Yes Receive JWT token Send request withAuthorization: Bearer token Tokenvalid? No 401 Unauthorized Yes Choose an action View profile · Add / View / Update / Delete workoutSearch workouts · AI recommendation · AI insights
Controller → ServiceMongoose → MongoDB Atlas Gemini AI Service→ Google Gemini API Standard JSON response → End
Errors at any stage are caught by thecentralized error handler and returned as JSON.
Figure 3: FitTrack AI – User Flow Diagram

9.4 MVC Architecture

The code follows the Model–View–Controller idea, adapted to a REST API. The **Model** is the Mongoose models (User and Workout) that represent the data in MongoDB Atlas. The **Controller** receives the request and coordinates the work. Because FitTrack AI has no user interface, the **View** is the JSON response returned to the client. Routes, authentication middleware and services are additional layers around MVC: routes direct requests, the middleware protects them, and services hold the business logic (including the Gemini AI service).

FitTrack AI – MVC Architecture Diagram
ClientThunder Client / Postman Routes + Authentication Middlewareroute match · JWT verification CONTROLLER (C)validate input · coordinate · respond Servicesbusiness logic · Gemini AI service
MODEL (M)Mongoose User · Workout MongoDB AtlasUsers and Workouts Google Gemini APIrecommendation / insight VIEW (V)JSON response (no HTML pages)
JSON response to client calls returns result Model View Controller Service layer AI
Routes, middleware and services are extra layers around the classic MVC pattern.
Figure 4: FitTrack AI – MVC Architecture Diagram

10. System Features

11. Implementation

The Phase 5 project setup and development work is summarized below. It should be checked against the actual project code.

11.1 Express.js Backend

`server.js` loads the environment variables, connects to MongoDB Atlas and starts the server. The Express application (`app.js`) applies the JSON body parser, CORS and Morgan request logging, mounts the route files and registers the centralized error handler last.

11.2 Routes, Middleware, Controllers and Services

- **Routes:** authentication (`/api/auth`), workout (`/api/workouts`) and AI (`/api/ai`) routes map URLs and methods to controller functions.
- **Middleware:** the authentication middleware verifies the Bearer JWT and attaches the user to the request; the error-handling middleware formats all errors.
- **Controllers:** read and validate input, call services and return the JSON response.
- **Services:** contain the business logic, including workout queries limited to the logged-in user and the Gemini AI service.

11.3 Mongoose Models and MongoDB Atlas

The **User** model has `name`, `email` (unique), `password` and timestamps. The **Workout** model has `workoutName`, `category` (validated), `duration`, `caloriesBurned`, `workoutDate`, `user` (reference to User) and timestamps. Mongoose connects to MongoDB Atlas using the connection string from the environment.

11.4 bcrypt.js and JWT

Before a user is saved, the password is hashed with `bcrypt.js`. At login the entered password is compared to the stored hash. After successful login, a JWT signed with the JWT secret is returned; the authentication middleware verifies it on protected routes.

11.5 Gemini AI Service

The Gemini AI service receives validated input from the AI controller, builds a prompt, sends it to the Google Gemini API with the API key from the environment, and returns the generated text to the controller, which sends it in the standard JSON format. The two AI endpoints use the input fields supplied in the request body: `age`, `fitnessGoal` and `experience` for recommendations, and `totalWorkouts`,

`averageDuration` and `totalCaloriesBurned` for insights. Failures of the service go to the centralized error handler.

FitTrack AI – Gemini AI Integration Diagram Client request JSON body with the AI input fields Route + Authentication Middleware /api/ai/recommendation · /api/ai/insights AI Controller check that required fields are present Gemini AI Service build a prompt from the validated input Google Gemini API authorized by the key in GEMINI_API_KEY Gemini AI Service read the generated text · return it AI Controller wrap in the standard JSON response Client receives the result AI recommendation / fitness insight Centralized Error Handler JSON error response (no key or stack trace) missing input AI failure Client Authentication Controller AI Response Error path (dashed) Figure 5: FitTrack AI – Gemini AI Integration Diagram

12. API Documentation Summary

Full details for each endpoint are in Part B.

13. Testing

The API is tested manually with Thunder Client / Postman using black-box test cases (the Phase 6 test plan). The plan covers:

Status of testing: the test cases are defined, but their actual results must be recorded during execution. This report therefore does not state how many tests passed or failed. After execution, insert the final Passed / Failed counts from the Phase 6 summary table here.

14. Results / Output

The expected output of each module is listed below. Actual outputs must come from the working project and are to be shown with screenshots captured from Thunder Client / Postman.

Screenshot evidence: [Insert screenshots from Thunder Client / Postman here – to be recorded during execution]

15. Benefits

- Keeps workout records in one structured, searchable place.

- Protects each user's data with password hashing, JWT authentication and per-user workout ownership.
- Adds AI recommendations and insights to the tracked data.
- The modular design makes the code easier to understand, test and extend.
- Any client that can call REST APIs can use the same backend.
- Cloud services (MongoDB Atlas) remove the need for a local database installation.

16. Limitations

- The AI features depend on the external Google Gemini service and its availability.
- An internet connection is required for MongoDB Atlas and the Gemini API.
- The project is a backend/API implementation; it has no graphical user interface.
- AI-generated recommendations and insights are general fitness guidance and not a substitute for advice from a qualified professional.
- The AI output can vary between requests, so the responses cannot be predicted exactly.

17. Future Scope

The following are possible future enhancements. None of them is implemented in the current project.

- A dedicated web frontend for the API.
- A mobile application.
- Advanced analytics and progress charts.
- Additional fitness metrics.
- Deeper AI personalization.

18. Conclusion

FitTrack AI is a REST API backend that combines secure workout tracking with AI-generated fitness guidance. Users can register, log in with JWT-based authentication, manage and search their workouts in MongoDB Atlas, and request recommendations and insights from Google Gemini. The layered design (routes, middleware, controllers, services and models), validation, centralized error handling and standard JSON responses make the API consistent and easy to maintain. The project has been documented from design to testing, and testing evidence must still be recorded during execution. The main limitations are its dependence on external services and the absence of a frontend, which are natural directions for future work.

Conventions. Requests and responses use JSON. Protected endpoints need the header `Authorization: Bearer <your-jwt-token>`. All values in the examples are placeholders, not real data. The response structure shown (`success, message, data`) is illustrative; check the exact fields against your implementation. Errors are returned by the centralized error handler as JSON with an appropriate status code.

C.1 Prerequisites

- A computer running Windows, Linux or macOS with an internet connection
- A code editor such as Visual Studio Code
- A MongoDB Atlas account and a Google Gemini API key
- Thunder Client (VS Code extension) or Postman
- Git (optional, for cloning the project)

C.2 Install Node.js

Download the LTS version from the official Node.js website and install it. npm is installed with Node.js. Check the installation:

C.5 Configure Environment Variables

Create a file named `.env` in the project root folder. Use your own values in place of the placeholders. Never share this file or commit it to GitHub.

Use the variable names your code reads (for example `MONGODB_URI` here; it must match the name in the project's database configuration).

C.6 Configure MongoDB Atlas

1. Create a MongoDB Atlas account and a cluster.
2. Create a database user with a username and password.
3. In Network Access, allow your IP address.
4. Use the cluster's Connect option to copy the connection string, replace the password placeholder with the database user's password, and store it in `MONGODB_URI`.

C.7 Configure the Gemini API

Create a Gemini API key by following the Google Gemini API documentation (see Part D), and store it in `GEMINI_API_KEY`. Keep the key private.

C.8 Start the Server



Use the script defined in the project's `package.json`. On success the console shows that the server is running and that MongoDB is connected (the exact messages depend on the code).

C.9 Test the Root API

In Thunder Client / Postman, send `GET http://localhost:5000/` (use your PORT value). A successful reply confirms that the server is running; the response text depends on the project code.

Using Thunder Client / Postman

1. Create a new request and choose the HTTP method (GET, POST, PUT or DELETE).
2. Enter the URL, for example `http://localhost:5000/api/auth/login`.
3. For POST and PUT, open **Body** → **JSON** and enter the request body.
4. For protected endpoints, open **Auth** → **Bearer** (or the Headers tab) and paste the token from the login response.
5. Click **Send** and check the status code and JSON response.
6. Optionally save the token as an environment variable (for example `token`) and use it as `{{token}}`.

C.10 Authentication Testing

1. `POST /api/auth/register` with name, email and password. Expect 201.
2. `POST /api/auth/login` with the same email and password. Expect 200 and a token. Copy the token.
3. `GET /api/auth/profile` with the Bearer token. Expect 200 and your profile. Without the token, expect 401.

C.11 Workout API Testing

1. `POST /api/workouts` with a valid workout body. Expect 201.
2. `GET /api/workouts` to list workouts. Expect 200.

3. `GET /api/workouts/:id` using an ID from the previous response. Expect 200.
4. `PUT /api/workouts/:id` with changed fields. Expect 200.
5. `GET /api/workouts/search?q=<keyword>`. Expect 200.
6. `DELETE /api/workouts/:id`. Expect 200.

C.12 AI API Testing

1. `POST /api/ai/recommendation` with `age`, `fitnessGoal` and `experience`. Expect 200 and a recommendation.
2. `POST /api/ai/insights` with `totalWorkouts`, `averageDuration` and `totalCaloriesBurned`. Expect 200 and insights.
3. Remove one required field from either request. Expect 400.

AI output may be different each time. Treat it as general fitness guidance.

Official documentation and resources. Add the date you accessed each source if your college requires it.

Phase 7 Conclusion

This documentation consolidates the project requirements, design, implementation, testing and usage information of FitTrack AI into a structured final project document. It presents the system as it is implemented, a REST API backend with authentication, workout management and search, and Gemini AI recommendations and insights. It also gives the API reference, the installation guide and the references needed to understand, run and test the project. Results that depend on running the project (test outcomes and screenshots) are marked to be recorded during execution, so that the final report contains only genuine evidence.